

Technical Manual

C++ → WebAssembly

→ GitHub Pages

Compilation, binary format, publishing and execution in the browser

Case study: Hands-on 1 (lexical analysis, DFA and Moore machine)

Alondra Goretti Gonzalez Martinez · Emiliano Barraza Cervantes

Professor: José Antonio Aviña Méndez

Version 1.0 · October 2026

Contents

1. Purpose, scope and big picture	3
2. Background: what WebAssembly is and why it works	4
3. Anatomy of the real binary (hands_on1.wasm)	5
4. Setting up the compiler (Emscripten SDK)	7
5. Preparing the C++ source for the browser	8
6. Compiling step by step	10
7. How the glue loads and starts the binary	12
8. index.html: connecting the interface to the engine	13
9. Testing locally before publishing	15
10. Git and GitHub: publishing the files	17
11. GitHub Pages: how the binary is served	19
12. What happens when someone opens the URL	21
13. How the practice was built (procedure)	22
14. Troubleshooting	23
Appendix A. Command cheat sheet	24
Appendix B. Glossary	24

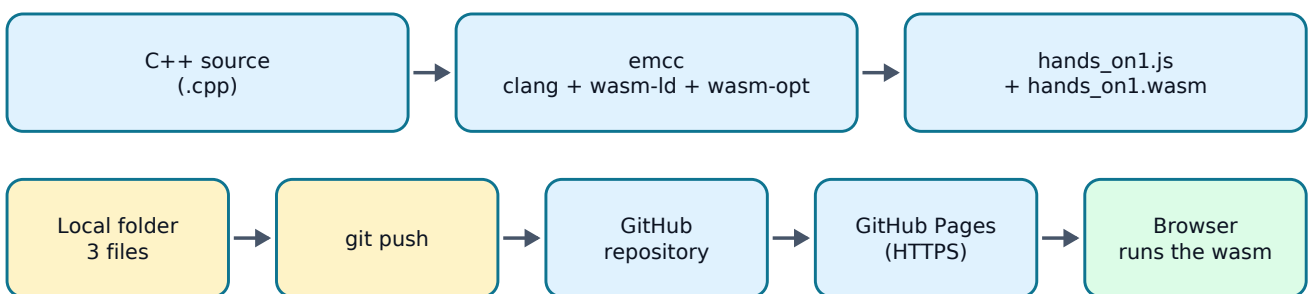
Tip: Everything in this manual that describes *your* files (sizes, sections of the binary, exported functions, test outputs) was measured on the delivered archive **hands_on1.zip**. Generic commands (installing Emscripten, Git, GitHub Pages) are the standard procedures; the compiler itself was not re-run to produce this document, so compare the numbers you get with yours.

1. Purpose, scope and big picture

This manual explains, step by step and with technical detail, how the C++ engine of the Hands-on 1 practice becomes a **binary WebAssembly module** that any browser can run, how that binary is **published through GitHub**, and how **GitHub Pages** delivers it so that it executes on the visitor's computer. It does not re-explain the automaton or the Moore machine (see the project README); it explains the *process and the machinery around them*.

1.1 The idea in one paragraph

The C++ source is compiled **once**, on the developer's machine, by the Emscripten compiler (**emcc**). The result is two files: a binary **hands_on1.wasm** (the program) and a text file **hands_on1.js** (the "glue" that loads it). Together with **index.html** they are ordinary static files. They are uploaded to a GitHub repository and GitHub Pages serves them over HTTPS. When a person opens the page, **their browser** downloads the three files, validates and compiles the binary to native machine code, and runs it. **No server executes C++**: GitHub only hands out files.



1.2 Inventory of the practice

File	Size (bytes)	Role	Publish?
HandsON1_gonzalez_martinez.cpp	10,369	Source code of the whole engine (DFA, instruction object, Moore, outputs, wasm interface).	Recommended
hands_on1.wasm	145,029	The compiled binary. Executed by the browser.	Required
hands_on1.js	22,694	Emscripten glue: loads and instantiates the .wasm, exposes the exports.	Required
index.html	13,258	User interface; talks to the wasm through the glue.	Required
README.md	10,756	Technical record of the solution.	Recommended
main.wasm	265,885	A different/earlier build. Not referenced by index.html nor by hands_on1.js.	No

2. Background: what WebAssembly is and why it works

2.1 The problem: browsers cannot run your g++ executable

When you compile with `g++` you get machine code for one processor family (x86-64, ARM...) and one operating system (Windows PE, Linux ELF...). A web page must work on any device, so browsers only execute **JavaScript** and **WebAssembly (wasm)**. WebAssembly is a **portable binary instruction format** for a small, safe virtual machine. It is not tied to any processor: each browser translates it to the native instructions of the visitor's CPU.

2.2 Key properties

- **Binary and compact:** a module starts with the 4 magic bytes `00 61 73 6D` (the text `"\0asm"`) and a version number; it is designed to be decoded and compiled very fast.
- **Typed and validated:** only four value types exist (`i32`, `i64`, `f32`, `f64`). The browser *validates* the whole module before running a single instruction; an invalid file is rejected.
- **Sandboxed linear memory:** the program sees one contiguous byte array (its "linear memory"). C++ pointers are just integer offsets into it. The module cannot read the page's memory or the user's disk.
- **No built-in I/O:** `wasm` cannot touch the DOM, print, read files or call the OS. Everything external arrives through *imports* that the host (JavaScript) supplies.
- **Near-native speed:** engines compile `wasm` to machine code ahead of execution, so it is far faster than interpreting.

2.3 Who does what

Piece	Responsibility in this practice
C++ code (compiled)	All logic: DFA, tokens, instruction object, Moore, JSON output.
<code>hands_on1.wasm</code>	The compiled logic + the C/C++ runtime libraries it needs (<code>libc</code> , <code>libc++</code>).
<code>hands_on1.js</code> (glue)	Downloads the <code>.wasm</code> , builds the imports, creates/links memory, runs static constructors, exposes <code>Module._h1_*</code> .
<code>index.html</code>	UI. Sends text to the <code>wasm</code> , receives JSON, draws the graph, tokens, object, Moore and registers.
Browser	Validates, compiles and runs the <code>wasm</code> in a sandbox; runs JavaScript.

2.4 Inside the compiler: what emcc really does

Emscripten is a toolchain, not a single program. `emcc` is a driver that behaves like `gcc` but orchestrates several tools:

1. **clang front-end:** parses C++17, produces LLVM intermediate code and then a WebAssembly object file (target `wasm32`).
2. **wasm-ld (linker):** joins your object with the pre-built system libraries (a C library and `libc++`). This is why a small program that uses `iostream`, `std::map` and `std::string` ends up with hundreds of functions (592 in this build).
3. **Binaryen wasm-opt:** optimizes and shrinks the binary when you pass `-O2` or higher.
4. **JS glue generator:** writes `hands_on1.js` tailored to what the program imports/exports, plus the loader code for the browser or Node.

3. Anatomy of the real binary (hands_on1.wasm)

The numbers below were obtained by parsing the delivered `hands_on1.wasm` byte by byte. They show what "the binary" actually is.

3.1 Header and sections

The file begins with the 8-byte header `00 61 73 6D 01 00 00 00`: the magic number (`\0asm`) and version 1. After it come *sections*, each stored as *id*, *size*, *content*:

Section	Bytes	%	Meaning
type	393	0.27	Function signatures used by the module (e.g. "(i32) → i32").
import	185	0.13	7 things the host must provide (see 3.3).
function	594	0.41	Which signature each of the 592 functions has.
table	7	0.00	Indirect-call table (virtual calls, function pointers).
memory	6	0.00	Linear memory: 256 pages, min = max (see 3.2).
global	8	0.01	Global variables (e.g. the C stack pointer).
export	278	0.19	What the module offers to JavaScript (see 3.4).
element	610	0.42	Initial contents of the indirect-call table.
code	131,323	90.55	The compiled bodies of the 592 functions: 90.5 % of the file.
data	11,589	7.99	35 data segments: string literals (error messages, JSON keys, state names) copied into memory at start-up.
(framing)	36	0.02	8-byte header + id/size bytes of every section.
Total	145,029	100	

Tip: Almost all of the weight is **code**: it is mostly the C++ standard library (iostream, string, map, set, optional, formatting) that was linked in, not your ~10 KB of source. This is normal for C++ compiled to wasm and explains why the binary is 145 KB.

3.2 Linear memory

The memory section declares **256 pages**. A wasm page is 64 KiB, so the module owns **16 MiB**. Minimum and maximum are both 256, therefore the memory is **fixed**: it cannot grow. That is Emscripten's default when `-sALLOW_MEMORY_GROWTH` is not used. Our program stores only the input line and one JSON string, so 16 MiB is ample; a program that allocates a lot would abort with an out-of-memory error unless compiled with growth enabled.

3.3 Imports (what the host must provide)

Import	Why it is there
<code>__cxa_throw</code>	C++ exception throwing used inside libc++ code paths; implemented by the glue.
<code>emscripten_memcpy_big</code> , <code>emscripten_resize_heap</code>	Fast bulk copy and heap-growth request (the latter fails here because memory is fixed).
<code>abort</code>	Called when the runtime hits an unrecoverable error.
<code>environ_sizes_get</code> , <code>environ_get</code>	C runtime start-up asks for environment variables (the glue answers with an empty set).
<code>strftime_l</code>	Locale/time formatting pulled in by iostream.

3.4 Exports (what JavaScript can call)

Export	Kind	Meaning
h1_limpiar, h1_byte, h1_procesar, h1_leer	function	Our interface (section 5). Present thanks to EMSCRIPTEN_KEEPALIVE .
analizar	function	Older string-based entry point kept for compatibility (needs a pointer).
memory	memory	The 16 MiB linear memory.
__wasm_call_ctors	function	Runs C++ static constructors (builds the tables R , ACCEPTA).
__indirect_function_table, __errno_location, stackSave / stackRestore / stackAlloc, __cxa_is_pointer_type, dynCall_*	various	Runtime plumbing generated by Emscripten (stack handling, errno, indirect calls). Not used directly by index.html.

3.5 A 41-byte module you can read completely

To demystify the format, here is a hand-assembled module exporting `add(a, b)`. It was run and returns `add(2,3) = 5`.

```
00 61 73 6D 01 00 00 00      header: magic "\0asm" + version 1
01 07 01 60 02 7F 7F 01 7F   type section: 1 type = func(i32,i32) -> i32
03 02 01 00                  function section: 1 function, uses type 0
07 07 01 03 61 64 64 00 00   export section: "add" (bytes 61 64 64) = function 0
0A 09 01 07 00 20 00 20 01 6A 0B code section: local.get 0, local.get 1, i32.add, end
```

```
const bytes = new Uint8Array([0x00,0x61,0x73,0x6d,1,0,0,0, 1,7,1,0x60,2,0x7f,0x7f,1,0x7f, 3,2,1,0,
7,7,1,3,0x61,0x64,0x64,0,0, 0x0a,9,1,7,0,0x20,0,0x20,1,0x6a,0x0b]);
const { instance } = await WebAssembly.instantiate(bytes);
console.log(instance.exports.add(2, 3)); // 5
```

The instructions `20 00, 20 01, 6A` are a *stack machine*: push parameter 0, push parameter 1, add the two top values. Your `hands_on1.wasm` is the same format, only with 592 functions instead of one.

3.6 hands_on1.wasm versus main.wasm

Aspect	hands_on1.wasm	main.wasm
Size	145,029 B	265,885 B
Our exports	h1_limpiar, h1_byte, h1_procesar, h1_leer, analizar	only analizar
Extra imports	none for output	fd_write, fd_close, fd_seek (console/stdio)
Referenced by index.html / hands_on1.js	Yes (name is written inside the .js)	No

Conclusion: `main.wasm` is a different build and the page never loads it. If it were renamed to `hands_on1.wasm`, the page would show `the .wasm does not bring h1_procesar` because that export does not exist in it.

4. Setting up the compiler (Emscripten SDK)

4.1 Requirements

- Git and Python 3 available in the terminal (the SDK installer is a Python script).
- Internet access during installation (it downloads clang, Binaryen, Node.js and the Emscripten sources). Several GB of disk are advisable.

4.2 Installation (emsdk)

```
git clone https://github.com/emscripten-core/emsdk.git
cd emsdk
./emsdk install latest           # Windows (cmd/PowerShell): emsdk install latest
./emsdk activate latest         # emsdk activate latest
source ./emsdk_env.sh           # Windows cmd: emsdk_env.bat PowerShell: .\emsdk_env.ps1
emcc --version                   # prints the Emscripten version if everything is in PATH
```

Note: The last `emsdk_env` step only affects the **current** terminal. If you open a new terminal and get `emcc: command not found`, run it again (or add the paths permanently).

4.3 Smoke test before touching the project

```
// hello.cpp
#include <iostream>
int main() { std::cout << "hola wasm\n"; }

emcc hello.cpp -o hello.js
node hello.js           # prints: hola wasm (Node runs the .js, which loads hello.wasm)
```

4.4 g++ versus emcc

Aspect	g++	emcc
Target	Your CPU + OS	wasm32 (portable)
Output	Executable (.exe / ELF)	.wasm + .js (or .html)
Who runs it	The operating system	Browser or Node.js
Entry point	<code>main()</code>	Exported functions (this project) or <code>main()</code>
Typical use here	Quick CLI tests	The web version

5. Preparing the C++ source for the browser

5.1 One source, two builds: `#ifdef __EMSCRIPTEN__`

Emscripten defines the macro `__EMSCRIPTEN__` automatically. The practice uses it to pick the entry point at compile time:

```
#ifdef __EMSCRIPTEN__
// ... exported functions for JavaScript (section 5.2) ...
#else
int main(int argc, char** argv) {      // g++ build: command-line program
    bool js = false; string in;
    for (int i = 1; i < argc; ++i) { string a = argv[i]; if (a == "--json") js = true; else in += ... }
    if (in.empty()) getline(cin, in);
    Res r = procesar(in);
    if (js) cout << aJson(r) << "\n"; else imprimir(r);
}
#endif
```

Both builds share exactly the same engine (`procesar()`, `aJson()`), so what you test in the console is what runs in the browser.

5.2 The exported interface

```
static string g_in, g_out;                // state that lives as long as the wasm instance
extern "C" {
    EMSCRIPTEN_KEEPALIVE void h1_limpiar() { g_in.clear(); }
    EMSCRIPTEN_KEEPALIVE void h1_byte(int b) { g_in.push_back((char)b); }
    EMSCRIPTEN_KEEPALIVE int h1_procesar() { g_out = aJson(procesar(g_in)); return (int)g_out.size(); }
    EMSCRIPTEN_KEEPALIVE int h1_leer(int i) { return (unsigned char)g_out[i]; }
    EMSCRIPTEN_KEEPALIVE const char* analizar(const char* s) { // older, string-based
        static string out; out = aJson(procesar(s)); return out.c_str(); }
}
```

Piece	Why it is needed
<code>extern "C"</code>	C++ normally <i>mangles</i> function names (e.g. <code>_Z10h1_procesarv</code>) to support overloading. <code>extern "C"</code> keeps the plain name <code>h1_procesar</code> so JavaScript can find it.
<code>EMSCRIPTEN_KEEPALIVE</code>	Tells the compiler/linker not to delete the function as "unused" and to export it. Without it, only <code>main</code> would survive. From <i>emscripten.h</i> .
<code>g_in / g_out</code>	Static globals persist between calls, so JavaScript can fill the input in several calls and read the output later.
<code>h1_procesar()</code>	Runs the whole pipeline and returns the length of the JSON so JS knows how many bytes to read.

5.3 Why only integers cross the boundary

A JavaScript string cannot be handed to C++ directly: wasm functions only receive numbers. To pass text you normally need **allocation helpers** (`malloc`, `stringToUTF8`) and **heap views** (`HEAPU8`) from the Emscripten runtime, and those are only available if you request them with link flags (`-sEXPORTED_RUNTIME_METHODS=...`, `-sEXPORTED_FUNCTIONS=...`). The practice avoids that dependency: JavaScript sends the text **one byte at a time** (`h1_byte`) and reads the JSON **one byte at a time** (`h1_leer`). Only `i32` values cross, so the interface works with *any* emcc flags. The source comment says exactly that: it does not depend on `cwrap`, `ccall`, `malloc` or `HEAPU8`.

5.4 The alternative: `cwrap`

Compiling with `-sEXPORTED_RUNTIME_METHODS=cwrap` adds the helper `Module.cwrap`, which wraps a C function (including string conversion) in a normal JS function:

```
const analizar = Module.cwrap('analizar', 'string', ['string']);  
const json = analizar("MOV AL, 6");      // simpler JS, but depends on the link flag
```

Note: In the delivered `hands_on1.js` the word `cwrap` does not appear, so that build was made **without** that flag. The README command still lists it. This is harmless for the page (it uses the integer functions) but you should make README and build command agree.

6. Compiling step by step

6.1 First, test natively with g++

Always validate the logic before involving the browser: errors are far easier to read in a terminal.

```
g++ -std=c++17 -O2 HandsON1_gonzalez_martinez.cpp -o hands_on1
./hands_on1 "MOV AL, 6"          # text report
./hands_on1 --json "STO 8"       # the same JSON the web page receives
```

Measured result for `MOV AL, 6`: tokens `MOV, REGISTRO("AL"), COMA, NUMERO("6")`; object `{operacion:"MOV", registros:["AL"], direccion:6}`; microoperations `MAR ← 6; MBR ← M[MAR]; AL ← MBR`. For `MOV AX, 6`: invalid, "registro inválido: MOV solo admite AL o BL".

6.2 Compile to WebAssembly

```
emcc HandsON1_gonzalez_martinez.cpp -std=c++17 -O2 -o hands_on1.js
```

Because of `-o hands_on1.js`, emcc writes **two** files in the current folder: `hands_on1.js` and `hands_on1.wasm` (same base name). Check them:

```
ls -l hands_on1.js hands_on1.wasm      # Windows: dir hands_on1.*
# expected in this project: hands_on1.js ~22.7 KB, hands_on1.wasm ~145 KB
```

6.3 The flags, one by one

Flag	What it does	Used in the practice?
<code>-std=c++17</code>	Selects the language standard. Required because the code uses <code>std::optional</code> .	Yes
<code>-O2</code>	Optimization level: speed and size. Also triggers Binaryen's <code>wasm-opt</code> . (<code>-O0</code> none, <code>-O1</code> , <code>-O2</code> , <code>-O3</code> more aggressive, <code>-Os</code> / <code>-Oz</code> smaller.)	Yes
<code>-o hands_on1.js</code>	Output name; its extension chooses the output kind: <code>.js</code> → <code>js+wasm</code> , <code>.html</code> → <code>js+wasm+HTML shell</code> , <code>.wasm</code> → <code>wasm only (standalone)</code> .	Yes
<code>-sEXPORTED_RUNTIME_METHODS=cwrap</code>	Exposes the JS helper <code>Module.cwrap</code> .	In README only; not in the delivered build
<code>-sEXPORTED_FUNCTIONS=_name,...</code>	Exports functions without needing <code>EMSCRIPTEN_KEEPALIVE</code> .	No (<code>KEEPALIVE</code> used)
<code>-sALLOW_MEMORY_GROWTH=1</code>	Lets memory grow beyond the initial 16 MiB.	No (memory is fixed)
<code>-sMODULARIZE=1</code> <code>-sEXPORT_NAME=X</code>	Wraps the glue in a factory function instead of a global <code>Module</code> .	No
<code>-sSINGLE_FILE=1</code>	Embeds the <code>wasm</code> inside the <code>.js</code> as base64 (one file, ~33 % bigger).	No
<code>-g</code> / <code>-sASSERTIONS=1</code>	Debug info and extra runtime checks while developing.	No

6.4 Names must match

- The glue contains the literal `wasmBinaryFile="hands_on1.wasm"`. If you rename the `.wasm`, the page breaks; recompile with another `-o` instead of renaming.
- The source file in the archive is `HandsON1_gonzalez_martinez.cpp` but the comments and README say `hands_on1.cpp`. Use the real name in the command, or rename the file consistently.

- Whenever you change the C++ you must recompile **and** re-upload both `hands_on1.js` and `hands_on1.wasm` (they are a matched pair).

7. How the glue loads and starts the binary

This is what the generated `hands_on1.js` does, following the real code of the delivered file.

1. **Takes or creates Module:** `var Module = typeof Module != "undefined" ? Module : {}`. If the page defined `Module` *before* loading the script, those settings (such as `onRuntimeInitialized`) are kept. That is why `index.html` declares `var Module = {...}` first.
2. **Locates the binary:** `wasmBinaryFile = "hands_on1.wasm"` resolved relative to the page URL (`locateFile`).
3. **Builds the imports:** `info = {env: wasmImports, wasi_snapshot_preview1: wasmImports}` — the 7 functions listed in section 3.3.
4. **Downloads and compiles:** if `WebAssembly.instantiateStreaming` exists and the page is *not* opened from `file://`, it streams (compiles while downloading). Otherwise, or if streaming fails, it does `fetch(...).arrayBuffer()` followed by `WebAssembly.instantiate`. On a streaming failure it logs falling back to `ArrayBuffer` instantiation.
5. **Receives the instance:** `Module.asm = instance.exports`; takes `memory` and builds the heap views.
6. **Runs the constructors:** `__wasm_call_ctors` builds the static tables `R` and `ACCEPTA` and the other static objects.
7. **Announces readiness:** calls `Module.onRuntimeInitialized()` — in this project, `boot()`.

7.1 Export wrappers

For each export the glue defines a thin function. They bind lazily to the real export the first time they are called:

```
var _h1_procesar = Module["_h1_procesar"] = function() {
  return (_h1_procesar = Module["_h1_procesar"] =
    Module["asm"]["h1_procesar"]).apply(null, arguments);
};
// JS name has a leading underscore (_h1_procesar); inside the wasm it is plain (h1_procesar)
```

7.2 Streaming versus ArrayBuffer

Path	Requirements	Behavior
<code>instantiateStreaming</code>	HTTP(S) origin; header <code>Content-Type: application/wasm</code>	Compiles while downloading: faster start.
<code>fetch + instantiate</code>	HTTP(S) origin; any content type	Downloads everything, then compiles. Fallback path.
<code>file://</code>	—	Does not work: browsers block fetching local files from a page (the glue itself skips fetch for file URIs).

8. index.html: connecting the interface to the engine

8.1 Script order matters

```
<script>
/* ... UI code ... */
function boot(){ ... }                                // called when the wasm is ready
var Module = { onRuntimeInitialized: boot,
                onAbort: w => { $('st').textContent = '· WASM abortó: ' + w } };
</script>
<script src="hands_on1.js"                             // relative path, same folder
        onerror="$('st').textContent='· no se pudo cargar hands_on1.js ...'"></script>
```

`Module` is defined **before** the glue is loaded so that the glue picks up the callbacks. The relative path `src="hands_on1.js"` works both on `localhost` and on `https://user.github.io/repo/`.

8.2 boot() and the bridge function an()

```
function boot(){
  const M = Module;
  if (typeof M._hl_procesar !== 'function') throw new Error('el .wasm no trae hl_procesar: ...');
  const enc = new TextEncoder(), dec = new TextDecoder();
  window.an = s => {
    M._hl_limpiar();                                // 1. clear the C++ input buffer
    for (const b of enc.encode(s)) M._hl_byte(b);    // 2. text -> UTF-8 bytes -> wasm
    const n = M._hl_procesar();                      // 3. C++ runs; returns JSON length
    const u = new Uint8Array(n);
    for (let i = 0; i < n; i++) u[i] = M._hl_leer(i); // 4. read the JSON byte by byte
    return dec.decode(u);                            // 5. bytes -> JS string
  };
  $('st').textContent = '· listo'; run();
}
```

UTF-8 is used on both sides, so characters such as ← and accented letters travel correctly. `run()` then does `D = JSON.parse(an(input))` and the rest of the page only draws `D`.

8.3 The JSON contract

Field	Content	Used for
entrada, ok, error	Cleaned input, validity flag, error message.	Header and error panel
pasos[]	Each DFA step: character <code>c</code> , from state <code>de</code> , to state <code>a</code> .	Step player, graph highlighting
tokens[]	<code>tipo</code> , <code>lex</code> , <code>paso</code>	Token list that appears progressively
objeto	<code>operacion</code> , <code>registros</code> , <code>direccion</code> (or null if invalid)	Instruction-object panel
moore[]	<code>estado</code> , <code>salida</code> , <code>sig</code>	Moore list and datapath animation
afd	<code>aristas[]</code> , <code>acepta[]</code>	Draws the graph from the real transition table

Measured: the JSON for `MOV AL, 6` is 1,717 bytes, because it includes the full DFA edge list on every call. The graph on the page is therefore always built from the same table the C++ uses — it can never disagree with the engine.

Note: The numeric values in the registers panel ($AL=5$, $BL=7$, $M[d]=(7d+3) \bmod 100$) are computed by the JavaScript function `machine()`, only to illustrate. The wasm produces the *text* of each microoperation, not the arithmetic.

9. Testing locally before publishing

9.1 Serve the folder over HTTP

```
cd folder-with-the-three-files
python3 -m http.server 8000          # Windows: py -m http.server 8000
# open http://localhost:8000/index.html
```

Do not double-click `index.html`: a `file://` page cannot fetch `hands_on1.wasm` (section 7.2). Measured with Python 3.12's built-in server: `hands_on1.wasm` is delivered with `Content-Type: application/wasm` and `Content-Length: 145029`, so streaming compilation works.

9.2 Browser checks (DevTools, F12)

- **Network:** three requests with status 200: `index.html`, `hands_on1.js`, `hands_on1.wasm` (type *wasm*, ~145 kB).
- **Console:** no red errors. Type `typeof Module._h1_procesar` → it must answer `"function"`.
- **Page:** the subtitle changes from `"(cargando...)"` to `"· listo"`.

9.3 Headless test with Node.js (no browser)

The same glue runs in Node, which allows an automatic test of the real binary:

```
// test.js -> node test.js
const M = require('./hands_on1.js');
M.onRuntimeInitialized = () => {
  const enc = new TextEncoder(), dec = new TextDecoder();
  const an = s => { M._h1_limpiar(); for (const b of enc.encode(s)) M._h1_byte(b);
    const n = M._h1_procesar(), u = new Uint8Array(n);
    for (let i = 0; i < n; i++) u[i] = M._h1_leer(i); return dec.decode(u); };
  const j = JSON.parse(an("MOV AL, 6"));
  console.log(j.ok, j.tokens.map(t => t.tipo + ':' + t.lex).join(' '),
    j.moore.map(m => m.salida).join(' ; '));
  console.log(JSON.parse(an("MOV AX, 6")).error);
};
```

Output obtained with the delivered files:

```
true MOV:MOV REGISTRO:AL COMA:, NUMERO:6 MAR ← 6 ; MBR ← M[MAR] ; AL ← MBR ;
registro inválido: MOV solo admite AL o BL
```

9.4 Test matrix

Input	Expected result
MOV AL, 6	valid · MAR ← 6; MBR ← M[MAR]; AL ← MBR
MOV BL, 7	valid · MAR ← 7; MBR ← M[MAR]; BL ← MBR
ADD AL, BL	valid · ACC ← AL + BL
STO 8	valid · MAR ← 8; MBR ← ACC; M[MAR] ← MBR
END	valid · HALT ← 1
MOV AX, 6	invalid · invalid register: MOV only accepts AL or BL
MOV AL 6	invalid · missing comma between operands
ADD AL,	invalid · missing register BL
STO	invalid · missing memory address in STO

Input	Expected result
END 8	invalid · END takes no operands

Take a screenshot of each case in the browser for the "Evidence" column that the README marks as pending.

10. Git and GitHub: publishing the files

10.1 What to upload

Group	Files	Why
Required	index.html, hands_on1.js, hands_on1.wasm	The site itself. They must sit in the same folder (relative paths).
Recommended	HandsON1_gonzalez_martinez.cpp, README.md	Source and documentation: the repository should let anyone rebuild the wasm.
Not needed	main.wasm, hands_on1.zip	main.wasm is not used; the zip is only a delivery format, GitHub does not need it.

10.2 One-time setup

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
# On github.com: New repository -> name it (e.g. hands-on1) -> Public -> Create
```

Note: GitHub Pages is available for **public** repositories on the Free plan; private repositories need a paid plan (GitHub's "Pages limits" documentation).

10.3 Commit and push

```
cd folder-with-the-files
git init
git add index.html hands_on1.js hands_on1.wasm HandsON1_gonzalez_martinez.cpp README.md
git commit -m "Hands-on 1: C++ engine compiled to WebAssembly"
git branch -M main
git remote add origin https://github.com/<user>/<repo>.git
git push -u origin main
```

Command	What happens
git init	Creates the hidden <code>.git</code> folder: the local history database.
git add	Marks files to be included in the next snapshot (the "staging area").
git commit	Stores the snapshot, with a message, in the local history.
git branch -M main	Names the branch <code>main</code> (the one Pages will publish).
git remote add origin	Registers the GitHub URL under the nickname "origin".
git push -u origin main	Uploads the commits and links local <code>main</code> with <code>origin/main</code> .

Authentication: when pushing over HTTPS, GitHub asks for a **personal access token** (or the Git Credential Manager signs you in through the browser); account passwords are no longer accepted for Git operations. SSH keys are the alternative. Never write a token inside a file you commit.

10.4 Protecting the binary and line endings

Git detects binary files by looking for zero bytes (a wasm starts with `00 61 73 6D`), but you can make it explicit and avoid any newline conversion by adding a `.gitattributes` file:

```
*.wasm binary
*.js text eol=lf
```

The delivered text files use Windows line endings (CRLF); that is harmless for the browser, only cosmetic in diffs.

10.5 Updating the site later

```
# edit the .cpp -> recompile -> verify locally -> publish
emcc HandsON1_gonzalez_martinez.cpp -std=c++17 -O2 -o hands_on1.js
git add hands_on1.js hands_on1.wasm HandsON1_gonzalez_martinez.cpp
git commit -m "Describe the change"
git push
```

If you prefer the web interface: *Add file* → *Upload files* in the repository page works for small files like these.

11. GitHub Pages: how the binary is served

11.1 Enabling it

1. Open the repository on github.com → **Settings** → **Pages**.
2. Under *Build and deployment*, *Source*: **Deploy from a branch**.
3. Branch: **main**, folder: / (**root**) → **Save**.
4. Wait for the deployment (Actions tab shows "pages build and deployment"). The site appears at <https://<user>.github.io/<repo>/>.
5. Open <.../<repo>/index.html> (or just <.../<repo>/>, since index.html is the default document).

11.2 What Pages is (and is not)

GitHub Pages is **static hosting**: whenever you push to the source branch, the files of the source folder are published. It does not run your code on the server. When publishing from a branch it uses Jekyll by default to build the site; for plain files like ours Jekyll just copies them. You may add an empty file named `.nojekyll` to skip that step. Alternatively a custom GitHub Actions workflow can build and publish.

11.3 URL resolution: the project-site trap

A project site lives under the repository name: <https://user.github.io/hands-on1/>. The relative reference `src="hands_on1.js"` resolves to /hands-on1/hands_on1.js ✓. An absolute reference `src="/hands_on1.js"` would point to the root of the domain /hands_on1.js ✗ (404). The glue also resolves `hands_on1.wasm` relative to the page, so keep everything relative. Remember that GitHub servers are **case-sensitive**: `Hands_on1.wasm` is a different file from `hands_on1.wasm`.

11.4 Content type, compression and caching

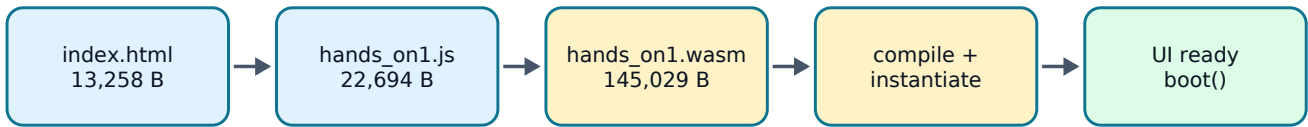
- **Content-Type**: Pages decides it from the file extension; you cannot set custom MIME types per file or per repository. A correct `application/wasm` enables streaming compilation. Verify it in DevTools → Network → `hands_on1.wasm` → Headers. If the type were wrong the glue would fall back to the `ArrayBuffer` path and still work.
- **Compression**: measured with `gzip -9` on the delivered files: `hands_on1.wasm` 145,029 → 55,171 bytes (38 %), `hands_on1.js` 22,694 → 6,820 bytes (30 %). Whether a given response is compressed is visible in the *Content-Encoding* header and in the Network column *Transferred* versus *Size*. You do not need to compress the files yourself.
- **Caching**: browsers and CDNs may keep old copies. After publishing a new build use a hard refresh (Ctrl+F5, or Cmd+Shift+R). GitHub's documentation suggests checking for build errors if changes do not appear after an hour.
- **HTTPS**: Pages serves over HTTPS, which also guarantees the page is a secure context.

11.5 Limits (GitHub documentation)

Limit	Value	Relevance here
Published site size	no larger than 1 GB	Ours: ~0.2 MB
Source repository	recommended 1 GB	Not an issue
Bandwidth	soft limit 100 GB / month	~175 KB per first visit
Builds	soft limit 10 per hour	Avoid pushing many times in a row
Deployment timeout	10 minutes	Not an issue

Note: Published Pages sites are public. Do not include credentials, personal data or anything private in the repository or the page.

12. What happens when someone opens the URL



#	Moment	What takes place
1	Connection	DNS resolves <i>user.github.io</i> ; a TLS (HTTPS) session is opened.
2	GET index.html	The browser downloads and parses the HTML and CSS; the first inline script runs and defines <code>Module</code> .
3	GET hands_on1.js	The second <code><script src></code> downloads and runs the glue. The glue starts fetching the binary.
4	GET hands_on1.wasm	The browser receives the bytes; it checks the magic number, validates every function and sections, and compiles to machine code (streaming if possible).
5	Instantiation	Memory of 16 MiB is created, the 35 data segments (strings) are copied in, the 7 imports are linked to JS functions, the exports become available.
6	Start-up	<code>__wasm_call_ctors</code> builds the static tables; the glue calls <code>onRuntimeInitialized</code> → <code>boot()</code> .
7	First analysis	<code>boot()</code> defines <code>an()</code> and calls <code>run()</code> with the default text <code>MOV AL, 6</code> .
8	Each click on Analizar	JS → bytes → <code>h1_*</code> → C++ → JSON → <code>JSON.parse</code> → SVG/DOM. Everything happens in the visitor's browser; GitHub is no longer involved.

12.1 Why it is safe to run

- The module is **validated** before execution; malformed or type-unsafe code is rejected.
- It only touches its own linear memory; out-of-range accesses trap instead of reading other memory.
- Its only way out is the 7 imports supplied by the glue (none gives file or network access).
- It runs under the browser's same-origin rules, like any script from that page.

13. How the practice was built (procedure)

This is the order in which the practice is organized, reconstructed from the delivered files and the README. It can be followed as a recipe.

#	Step	Where / output
1	Define the reference ISA (MOV, ADD, STO, END), registers and microoperations.	README §2
2	Design the DFA as an explicit transition table (state, character, next state) and the accepting states.	R, ACEPTA
3	Write the engine: sig, tipo, msg, procesar, Instr, class Moore.	.cpp
4	Add the two outputs: text report (imprimir) and JSON (aJson).	.cpp
5	Test in the console with g++ using valid and invalid inputs.	section 6.1, 9.4
6	Add the Emscripten interface inside <code>#ifdef __EMSCRIPTEN__</code> (integer-only functions).	section 5
7	Compile with emcc → hands_on1.js + hands_on1.wasm.	section 6.2
8	Write index.html: graph, tokens, object, Moore, datapath, player, challenge mode.	index.html
9	Serve locally and verify in the browser.	section 9
10	Push the three files (plus source and README) to GitHub and enable Pages.	sections 10–11
11	Document: README with test cases and evidence screenshots.	README §9

Tip: The archive contains two builds: main.wasm (only analizar) and hands_on1.wasm (with h1_*). Together with the source comment about "not depending on cwrap", this is consistent with the interface having been changed from a string-based call to the integer-only one. Only hands_on1.wasm is part of the final site.

14. Troubleshooting

Symptom	Probable cause	Fix
Page stays at "(cargando...)" or "no se pudo cargar hands_on1.js"	Wrong file name/case, file not uploaded, or wrong folder (Network shows 404).	Make names and case match exactly; keep the three files together.
Works on the server but not by double-click	file:// blocks loading the .wasm.	Use <code>python3 -m http.server</code> or GitHub Pages.
<code>expected magic word 3c 21 44 4f</code>	The server returned an HTML page (404) instead of the binary: bytes 3c 21 44 4f spell "<!DOCTYPE".	Fix the .wasm path/name.
<code>el .wasm no trae h1_procesar</code>	Old or different wasm (e.g. main.wasm renamed) or built from an old source.	Recompile the current .cpp and upload both .js and .wasm.
<code>falling back to ArrayBuffer instantiation</code>	Streaming failed (usually wrong Content-Type).	It still works; check the header if you want streaming.
Old version still showing after push	Cache, or deployment not finished.	Wait for the Actions run; hard refresh Ctrl+F5.
<code>emcc: command not found</code>	emsdk environment not loaded in this terminal.	Run <code>emsdk_env</code> again.
<code>Module.cwrap is not a function</code>	Built without <code>-sEXPORTED_RUNTIME_METHODS=cwrap</code> .	Use the integer <code>_h1_*</code> functions or add the flag.
Garbled characters (Ã©, â†)	Missing UTF-8 declaration.	Keep <code><meta charset="utf-8"></code> and the TextEncoder/TextDecoder pair.
Aborts with OOM on huge inputs	Memory is fixed at 16 MiB.	Compile with <code>-sALLOW_MEMORY_GROWTH=1</code> .
Pages gives 404 for the whole site	Pages not enabled, wrong branch/folder, or private repo on the Free plan.	Re-check Settings → Pages (section 11.1).

Appendix A. Command cheat sheet

```
# --- Compiler setup (once) ---
git clone https://github.com/emscripten-core/emsdk.git && cd emsdk
./emsdk install latest && ./emsdk activate latest && source ./emsdk_env.sh
emcc --version

# --- Native test ---
g++ -std=c++17 -O2 HandsON1_gonzalez_martinez.cpp -o hands_on1
./hands_on1 "MOV AL, 6" ./hands_on1 --json "STO 8"

# --- WebAssembly build ---
emcc HandsON1_gonzalez_martinez.cpp -std=c++17 -O2 -o hands_on1.js

# --- Local server ---
python3 -m http.server 8000 # http://localhost:8000/index.html

# --- Publish ---
git init && git add . && git commit -m "Hands-on 1" && git branch -M main
git remote add origin https://github.com/<user>/<repo>.git && git push -u origin main
# GitHub: Settings -> Pages -> Deploy from a branch -> main / (root)
```

Pre-delivery checklist

- The three files are in the same folder and the names match exactly (case included).
- `hands_on1.js` and `hands_on1.wasm` come from the **same** compilation.
- Local test with a server: 5 valid + 5 invalid inputs behave as in section 9.4.
- README commands use the real source file name and match the flags actually used.
- Evidence screenshots added to the README test tables.
- Pages URL opens in a private window (no cache) and shows "· listo".

Appendix B. Glossary

Term	Meaning
WebAssembly (wasm)	Portable binary instruction format executed by browsers in a sandbox.
Emscripten / emcc	Toolchain and compiler driver that turns C/C++ into wasm + JS glue.
LLVM / clang	Compiler infrastructure and C/C++ front-end used by Emscripten.
Binaryen / wasm-opt	Optimizer for wasm modules.
Glue (hands_on1.js)	Generated JS that loads and connects the wasm to the page.
Linear memory	The module's private byte array; pointers are offsets in it.
Import / Export	Function the host gives to the module / function the module gives to the host.
EMSCRIPTEN_KEEPALIVE	Macro that keeps and exports a function.
MIME type	Label (Content-Type) telling the browser what a downloaded file is; wasm → application/wasm.
GitHub Pages	Static hosting service that publishes a repository branch/folder over HTTPS.
Static constructors	C++ objects built before use; wasm runs them via <code>__wasm_call_ctors</code> .